Industrial ML Taxonomy: Regression & Anomaly Detection

Platform blueprint for industrial sensor and manufacturing data

---

0. Executive summary

Regression and anomaly detection (AD) are the two workhorse problem families in industrial ML — soft sensors, RUL, yield, and quality prediction on one side; fault detection, drift, and defect detection on the other. They look similar at the infrastructure level (same sensors, same time-series shape, same ingestion path) but diverge sharply at the modeling level (different objectives, different label regimes, different failure modes). A platform that tries to force both into one modeling pipeline will break; a platform that shares nothing will duplicate effort and drift out of sync.

Four principles drive every decision in this document:

1. **Label availability is the real fork in the road**, not the algorithm family. Regression almost always needs a continuous ground truth; AD usually has to work with little or no fault labels, which is why semi-supervised (train-on-normal) approaches dominate industrial AD.
2. **Time-series vs. tabular vs. multivariate sensor data changes validation strategy, feature engineering, and model class** — this cuts across both families, so treat it as an orthogonal axis, not a regression-only or AD-only concern.
3. **Infrastructure is shared, modeling logic is not.** Ingestion, schema validation, the feature store, the model registry, and the monitoring stack can be one shared system. Loss functions, thresholds, and evaluation metrics cannot.
4. **The scenario checklist is the actual deliverable.** Algorithms come and go; the list of real-world scenarios a platform must be able to express (RUL, soft sensor, sensor dropout, novel operating mode, collective drift, etc.) is what prevents a costly redesign six months in.

---

1. Regression hierarchy

1.1 Target types

Target type	Description	Example
Scalar continuous	Single numeric value, one prediction per record	Predicted outlet temperature
Multi-output / vector	Several correlated continuous targets predicted jointly	Predicting tensile strength, hardness, and thickness

Target type	Description	Example
		together
Time-to-event (survival-like)	Continuous target with censoring — asset may still be running when data is collected	Remaining Useful Life (RUL)
Horizon forecast	Target is the sensor's own future value at t+h	Load forecast 4 hours ahead
Ordinal-continuous hybrid	Continuous internally but reported/consumed as a bounded score	Inline quality score (0–100) mapped to a lab grade

1.2 Input data types

- Tabular process parameters (setpoints, batch recipe fields, static config)
- Time-series sensor streams — single-channel or multi-channel, uniform or mixed sample rate
- Spectral / vibration data (FFT-ready waveform segments)
- Vision-derived features (extracted embeddings or measurements from inline cameras)
- Static asset metadata (machine age, model, maintenance history, install date)
- Event and maintenance logs (work orders, alarms, operator notes)

1.3 Common industrial scenarios

- **Soft sensor / virtual sensor** — predicting a hard-to-measure or expensive-to-measure variable (e.g. composition) from cheap, fast sensors.
- **Remaining Useful Life (RUL) estimation** — predicting time-to-failure or time-to-maintenance from degradation signals.
- **Quality measurement prediction** — predicting a lab test result (destructive or slow) from inline process sensors, enabling real-time release decisions.
- **Yield prediction** — predicting batch or line yield from process conditions.
- **Energy / utility consumption prediction** — predicting power, steam, or compressed-air draw.
- **Cycle time / throughput prediction** — predicting how long a batch or unit operation will take.
- **Short-horizon forecasting** — demand, load, or consumption forecasting, which is regression against a shifted target.
- **Multi-output regression** — jointly predicting several correlated quality attributes to preserve their correlation structure, rather than training N independent models.

Algorithm	Best for	When to use	Strengths	Weaknesses
Linear / Ridge / Lasso / PLS	High-dimensional collinear sensor data (spectroscopy, chemometrics)	Small-to-medium data, interpretability required, strong multicollinearity	Fast, interpretable, stable with regularization	Misses non-linear interactions
Gradient boosted trees (XGBoost, LightGBM, CatBoost)	Structured tabular process data	Medium-to-large data, non-linear interactions, mixed feature types	Strong default baseline, handles missing values, feature importance built in	Needs care with temporal leakage, weaker on raw sequences
Random Forest	Robust baseline, noisy sensor data	Quick baseline, noisy or small data, need feature importance	Robust to outliers, low tuning effort	Less accurate than boosting on large clean data
Support Vector Regression (SVR)	Small, smooth-function data	Small datasets, smooth non-linear relationships	Works well with limited data	Scales poorly, sensitive to kernel/hyperparameter choice
Gaussian Process Regression	Small data needing uncertainty	Need calibrated confidence intervals, sparse data	Native uncertainty estimates	Computationally expensive beyond a few thousand rows
MLP (feed-forward neural net)	Tabular data with complex interactions	Large tabular datasets, non-linear feature interactions	Flexible, scales with data	Needs more data and tuning than trees, less interpretable
LSTM / GRU	Sequential regression where history matters	RUL, degradation trends, variable-length sequences	Captures long-range temporal dependency	Slower to train, needs sequence framing, prone to overfitting on short series
TCN (temporal	Sequential regression,	Long sequences, need faster	Parallel training, long	Less mature tooling than LSTM in some stacks

Algorithm	Best for	When to use	Strengths	Weaknesses
convolutional network)	parallelizable training	training than RNNs	effective receptive field	
Transformer / temporal fusion	Multiple correlated sensors, long sequences	Large fleets, many correlated channels, attention over time helps interpretability	Handles multivariate long sequences, attention aids explainability	Data-hungry, higher engineering cost
Survival regression (Cox PH, Weibull AFT)	Censored RUL data	Asset still operating when data is collected (right-censored)	Properly handles censoring, statistically principled	Less familiar tooling, assumes specific hazard structure
Physics-informed / hybrid models	Known physical model + sparse data	Well-understood physics but insufficient data for pure ML	Extrapolates better, needs less data	Requires domain physics expertise to build

1.5 Data assumptions

- Sensors sampled consistently, or a defined resampling/alignment strategy exists for mixed rates.
- Training data spans the full operating envelope the model will be asked to predict on — extrapolation outside it is unreliable.
- Labels (lab results, RUL ground truth) are timestamped correctly and lag-aligned with the sensor window that produced them.
- For classic tabular ML: no data leakage across time — train/validation split must respect chronology, not random shuffling.
- Missing data has a known pattern (random vs. systematic, e.g. sensor maintenance windows) that preprocessing accounts for.

1.6 Failure modes

- **Temporal leakage** — using future information (e.g. a rolling feature computed across the full series) to predict the past.
- **Extrapolation beyond training envelope** — new operating point, new product recipe, or new asset outside the training distribution.
- **Target drift** — the physical relationship between sensors and target shifts over time (equipment wear, recalibration, process change) without retraining.

- **Degenerate / stuck sensor** — a frozen or flat-lined sensor gets picked up as a spurious strong predictor.
- **Asset-specific overfitting** — a model trained on one machine fails to generalize across a fleet with slightly different dynamics.
- **Label scarcity for RUL/quality** — destructive testing or slow lab cycles mean very few true labels, forcing heavy reliance on proxy labels.

1.7 Evaluation metrics

- RMSE, MAE, MAPE, R^2 as baseline metrics.
- NRMSE (normalized RMSE) when comparing performance across assets or units with different scales.
- For RUL specifically: **asymmetric scoring** that penalizes late predictions more than early ones (e.g. the PHM08-style scoring function), since predicting failure too late is operationally worse than predicting it too early.
- For multi-output regression: report per-target metrics individually, plus one aggregate (e.g. mean NRMSE) — never collapse to a single number without the breakdown.

1.8 Preprocessing & feature engineering

- Resampling and time alignment across multi-rate sensors to a common clock.
- Rolling statistics: mean, std, min/max, slope over multiple window lengths.
- Lag features for known-delay relationships (e.g. sensor-to-lab-result lag).
- Spectral / FFT features for vibration and acoustic signals.
- Domain-specific degradation indices (e.g. cumulative wear proxies) where physical knowledge exists.
- Outlier clipping or winsorizing before scaling.
- Per-asset vs. global normalization — decide based on whether cross-asset comparability or single-asset precision matters more.
- Explicit handling of censored or missing labels rather than silent row-dropping.

1.9 Pipeline stages

Ingestion → time alignment/resampling → data validation → feature engineering → time-aware train/validation split → model training → evaluation → calibration (for confidence intervals where relevant) → deployment → monitoring.

---

2. Anomaly detection hierarchy

2.1 Types of anomalies

Type	Description
Point anomaly	A single observation that is anomalous relative to the rest of the data (a spike)
Contextual anomaly	Normal value in general, but abnormal given the current context (e.g. high temperature is normal in summer, anomalous in winter for the same setpoint)
Collective / sequence anomaly	Individual points look normal, but the pattern over a window is abnormal (a slow oscillation, a subtle regime change)
Drift-related anomaly	The data-generating process itself shifts over time (concept drift), which can either be the anomaly of interest or a source of false alarms if not modeled separately

2.2 Supervision levels

- **Supervised** — labeled fault and normal examples exist; framed as classification. Rare in industrial settings because faults are infrequent and expensive to label.
- **Semi-supervised (train-on-normal)** — the dominant industrial pattern. Model learns what "normal" looks like from a clean training set, then flags deviations. Doesn't require fault labels, only a curated normal period.
- **Unsupervised** — no labels of any kind; relies purely on statistical or structural properties of the data (density, distance, clustering) to flag rare points.

2.3 Input data types

Same categories as regression (tabular, time-series, spectral, vision, static metadata) — the same sensor infrastructure feeds both problem families.

2.4 Common industrial scenarios

- **Equipment fault detection** — bearing wear, pump cavitation, motor imbalance.
- **Sensor spike and dropout detection** — data-quality anomalies that must be distinguished from process anomalies.
- **Novelty detection** — flagging a genuinely new operating mode the model has never seen (new product, new recipe, new ambient condition).
- **Process deviation detection** — the process drifts outside its normal operating band without a discrete fault event.
- **Predictive maintenance early warning** — subtle multivariate deviation that precedes a known failure mode by hours or days.

- **Visual / quality defect detection** — anomalies in vision-derived features (surface defects, misalignment).
- **OT / cybersecurity anomaly** — anomalous command or network patterns on operational technology; same statistical toolkit, different feature space and much higher false-positive cost tolerance for security triage.

2.5 Algorithm guide by category

Method	Anomaly type fit	When to use	Strengths	Weaknesses
Rule / threshold-based (SPC charts: Shewhart, CUSUM, EWMA)	Point, drift	Domain SME thresholds are known, interpretability is critical, regulatory audit trail needed	Fully interpretable, easy to explain to operators	Misses multivariate and contextual anomalies, brittle to legitimate regime changes
Statistical / density (Z-score, Mahalanobis distance, Hotelling's T^2, GMM, KDE)	Point, contextual	Linear correlation structure among sensors is known and stable	Statistically grounded, fast to compute	Assumes a distribution shape, weak with strong non-linearity
Distance / cluster-based (k-NN distance, k-means distance, DBSCAN)	Point, novelty	Fast unsupervised baseline, moderate dimensionality	No distributional assumption, simple to implement	Struggles in high dimensions, sensitive to distance metric choice
Isolation Forest	Point, novelty	Fast unsupervised baseline across many features	Scales well, no distance computation, handles high dimensionality	Less effective on strongly sequential/contextual anomalies
One-Class SVM / SVDD	Point, novelty	Small clean training set, tight normal boundary needed	Works with limited normal data	Sensitive to kernel/hyperparameter tuning, scales poorly

Method	Anomaly type fit	When to use	Strengths	Weaknesses
Reconstruction-based (PCA residual, Autoencoder, VAE, LSTM-Autoencoder)	Point, contextual, collective	Non-linear multivariate relationships, sequential data	Captures complex multivariate structure, LSTM-AE captures temporal pattern	Needs enough clean normal data to learn reconstruction well, can be opaque
Forecasting-residual based (predict next value, flag large residual)	Point, contextual	Bridges regression and AD — useful when a good forecasting model already exists	Reuses regression infrastructure, intuitive	Residual threshold needs careful calibration, sensitive to forecast model drift
Supervised classifiers (RF, GBM, NN)	Point, collective	Enough labeled fault examples exist and fault modes are stable	Highest precision when labels are available	Requires labeled faults, which are rare and expensive
Changepoint detection (CUSUM, Bayesian online changepoint)	Drift	Monitoring for a regime shift rather than a point outlier	Detects gradual shifts other methods miss	Detection delay is inherent to the method, needs tuning of sensitivity
Matrix profile / discords	Collective / sequence	Long time-series where the shape of a subsequence matters more than any single value	Finds subtle repeating-pattern anomalies without labels	Computationally heavier on very long series

2.6 Data assumptions

- The "normal" training period genuinely represents typical operation across all valid regimes (shifts, seasons, product types) — not just one slice of it.
- For semi-supervised methods: the normal training set is contamination-free (no undetected faults baked into "normal").
- Enough history exists to characterize normal variability, including legitimate regime changes, so they aren't flagged as anomalies.

2.7 Failure modes

- **Contaminated normal set** — undetected faults inside what was assumed to be clean training data quietly raise the threshold.
- **Multiple valid operating modes not modeled** — a legitimate regime change gets flagged as an anomaly (a major source of alarm fatigue).
- **Sensor drift mistaken for process anomaly** — gradual sensor calibration drift looks identical to a genuine slow-developing fault.
- **Alarm fatigue** — threshold set too sensitive, operators start ignoring alerts.
- **Silent model staleness** — the normal envelope shifts (new product mix, seasonal change) but the model isn't retrained, causing both missed detections and false alarms.
- **Boundary too tight or too loose** — one-class boundary methods (SVM/SVDD, Isolation Forest) need re-validation whenever the operating envelope changes.

2.8 Evaluation metrics

- Precision, recall, F1 — with explicit attention to class imbalance (anomalies are rare by definition).
- ROC-AUC and PR-AUC (PR-AUC is usually more informative given the imbalance).
- **Detection latency / time-to-detect** — often more important operationally than point-level precision.
- **False alarm rate per week** — the business-facing metric that determines whether operators trust the system.
- Point-adjust evaluation caveats — a common evaluation trick (crediting a whole anomalous segment as detected if any point in it was flagged) can inflate reported performance; state clearly which convention is used.
- For changepoint detection specifically: detection delay from the true changepoint.

2.9 Preprocessing & feature engineering

- Same alignment/resampling pipeline as regression.
- Residual features (from a forecasting or reconstruction model) as anomaly-scoring inputs.
- Rolling z-scores and rolling statistics for contextual anomaly detection.
- Wavelet / FFT features for vibration-based fault detection.
- Dimensionality reduction (PCA) before reconstruction-based modeling to control the effective feature space.
- Per-operating-mode normalization — normalize within each known regime rather than globally, when multiple valid modes exist.
- Careful label curation for the supervised branch — fault tagging protocol should be documented and consistent.

2.10 Pipeline stages

Ingestion → data validation → normal-state feature engineering → model training (on curated normal data for semi-supervised methods) → threshold calibration (e.g. based on a validation-set percentile) → evaluation → alerting/monitoring integration → retraining trigger via drift detection on the input feature distribution.

---

3. Shared pipeline architecture

3.1 End-to-end structure

Ingestion → schema validation → time alignment/resampling → data quality checks → feature engineering (shared library) → **[regression training / evaluation]** or **[AD training / threshold calibration]** → deployment (shared serving layer) → monitoring and drift tracking (shared) → retraining trigger.

(See the diagram above for the visual layout of this flow.)

3.2 Stages that can be shared

- Data ingestion and connector layer.
- Schema validation and data-quality checks.
- Time alignment / resampling utilities.
- The feature engineering library (rolling stats, lag features, spectral features, cross-sensor ratios) — same transforms, reused by both families.
- Experiment tracking (run metadata, parameters, artifacts).
- Model registry and versioning.
- Deployment / serving infrastructure.
- Monitoring infrastructure (metric collection, dashboards, alerting transport).
- The general retraining orchestration skeleton (the scheduler and trigger mechanism, not the retraining logic itself).

3.3 What goes into each stage

- **Data validation**: schema conformance, per-sensor range checks, missing-value rate thresholds, timestamp continuity/gap detection, duplicate-record detection, unit and sensor-identity consistency, cross-sensor correlation sanity checks (catches wiring/tagging errors).
- **Feature engineering**: the shared transform library described above, parameterized per use case rather than duplicated per model.
- **Model training**: shared training harness (data loaders, experiment logging) wrapping a family-specific objective — continuous loss for regression, reconstruction/density/margin objective for AD.

- **Evaluation**: family-specific metric modules (Section 1.7 and 2.8) computed through a shared evaluation harness so results land in the same tracking system.
- **Explainability**: SHAP or feature-importance tooling shared at the library level, but the interpretation differs — for regression it explains what drives the predicted value, for AD it explains what drives the deviation from normal.
- **Monitoring and drift tracking**: shared drift detection on input feature distributions (e.g. population stability index, KS-test), but the downstream action differs — for regression, performance decay triggers a retrain; for AD, drift can mean either "recalibrate the threshold" or "retrain the normal-state model," and the platform must distinguish which.

---

4. Separate pipeline requirements

These must **not** be unified, even though the surrounding infrastructure is shared:

- **Label handling** — regression needs a continuous ground-truth value; AD needs either a curated "normal period" definition (semi-supervised) or fault labels (supervised). These are structurally different data contracts, not variations of the same one.
- **Training objective / loss function** — continuous regression loss vs. reconstruction loss, density estimation, or margin-based objectives.
- **Threshold calibration** — unique to AD; regression has no equivalent concept.
- **Evaluation metric computation** — RMSE-family metrics vs. precision/recall/detection-latency metrics measure fundamentally different things.
- **Alerting and downstream action logic** — a regression prediction feeds a dashboard or a control loop; an AD flag feeds an alarm/ticketing workflow with its own escalation rules.
- **Retraining trigger semantics** — "performance decayed, retrain" (regression) vs. "input distribution drifted, recalibrate threshold or retrain the normal model" (AD) are different decisions requiring different signals.

---

5. Industrial considerations

Schema differences. Sensor naming, units, and tag structures vary by plant, line, and even by shift-to-shift configuration changes. A multi-tenant platform needs a canonical schema mapping layer (per-tenant tag registry) between raw ingestion and the shared feature engineering library — without it, every new site becomes a bespoke integration project instead of a configuration exercise.

Time-series vs. tabular vs. multivariate. Time-series data forces a chronological train/validation split (never random shuffling) and favors sequence-aware models (LSTM/TCN/Transformer) or careful windowed feature engineering. Tabular data (e.g. per-batch summary records) tolerates standard cross-validation. Multivariate sensor arrays

need correlation-aware feature engineering and often benefit from dimensionality reduction before modeling, especially for reconstruction-based AD.

Label availability. This is the single biggest driver of algorithm choice. Abundant labels → supervised regression and supervised AD classifiers become viable. Scarce labels (the industrial default, since lab tests and fault tagging are slow and expensive) → soft-sensor regression with proxy labels, and semi-supervised (train-on-normal) AD.

Unified vs. separate framework decision. The right pattern is "one platform, many pipelines": unify the infrastructure layer (ingestion, validation, feature store, deployment, monitoring) and keep the modeling layer (objective, threshold, evaluation, alerting) separate per family. Attempting a single unified modeling pipeline for both families is the most common design mistake in industrial ML platforms.

What must not be forced into a shared pipeline: loss functions, threshold logic, label schemas, evaluation metric modules, alerting/escalation logic.

What is safe to generalize: ingestion, schema validation, the feature transform library, the model registry, deployment/serving, and the monitoring/drift-detection infrastructure (with family-specific action logic layered on top).

---

6. Decision table for algorithm selection

Scenario	Data type	Label availability	Recommended approach	Fallback
Soft sensor / virtual sensor	Multivariate time-series	Continuous labels available (lagged)	Gradient boosted trees or LSTM if strong temporal dependency	Linear/PLS if highly collinear and small data
RUL estimation	Time-series, degradation trend	Censored continuous (asset may still be running)	Survival regression (Cox/Weibull) or LSTM with asymmetric scoring	GBM on hand-engineered degradation features
Quality prediction from inline sensors	Tabular + time-series	Sparse lab labels	GBM with careful lag alignment	PLS regression (chemometrics-style)
Yield prediction	Tabular (batch records)	Continuous, moderate volume	GBM or Random Forest	Linear regression baseline

Scenario	Data type	Label availability	Recommended approach	Fallback
Energy consumption prediction	Time-series	Continuous, abundant	GBM or TCN	Linear regression with rolling features
Equipment fault detection, known fault modes	Time-series / vibration	Some labeled faults	Supervised classifier (RF/GBM)	Semi-supervised reconstruction model if labels too few
Equipment fault detection, unknown/rare faults	Time-series / vibration	No fault labels	Semi-supervised Autoencoder or Isolation Forest	PCA residual + threshold
Sensor spike/dropout detection	Time-series, single or few channels	Usually none	Rule/threshold-based (SPC)	Z-score / rolling statistics
Novelty detection (new operating mode)	Multivariate tabular/time-series	None	One-Class SVM or Isolation Forest	Distance-based (k-NN)
Process deviation / drift monitoring	Multivariate time-series	None	Changepoint detection (CUSUM)	PSI/KS-test on features
Predictive maintenance early warning	Multivariate time-series	Little to none	LSTM-Autoencoder or forecasting-residual method	PCA reconstruction error
Visual/quality defect detection	Image-derived features	Some labeled defects	Supervised classifier on embeddings	One-class model on "good" embeddings only
Collective/sequence pattern anomaly	Long time-series	None	Matrix profile / discords	LSTM-Autoencoder

7. Checklist of scenarios that must not be missed

Regression

- [ ] Soft sensor / virtual sensor prediction
- [ ] RUL estimation with censored data handling

- [ ] Quality prediction with sparse, lagged lab labels
- [ ] Yield prediction
- [ ] Energy/utility consumption prediction
- [ ] Cycle time / throughput prediction
- [ ] Short-horizon forecasting framed as regression
- [ ] Multi-output regression preserving cross-target correlation
- [ ] Per-asset vs. fleet-wide generalization

Anomaly detection

- [ ] Point anomaly detection (spikes)
- [ ] Contextual anomaly detection (context-dependent thresholds)
- [ ] Collective / sequence anomaly detection (subtle pattern shifts)
- [ ] Drift-related anomaly / changepoint detection
- [ ] Equipment fault detection with and without labels
- [ ] Sensor spike and dropout detection (data-quality vs. process anomaly)
- [ ] Novelty detection for new operating modes
- [ ] Cluster-based anomaly detection
- [ ] Reconstruction-based anomaly detection (PCA, Autoencoder, LSTM-AE)
- [ ] Density-based anomaly detection
- [ ] Rule/threshold-based detection with SME-defined limits
- [ ] Multiple legitimate operating regimes explicitly modeled (not flagged as anomalies)

Platform-level

- [ ] Canonical schema mapping for multi-tenant/multi-site onboarding
- [ ] Chronological (not random) train/validation splitting wherever time-series data is involved
- [ ] Separate label contracts for regression vs. AD
- [ ] Separate threshold-calibration logic isolated from the shared training harness
- [ ] Drift monitoring wired to two distinct actions (retrain vs. recalibrate) depending on family
- [ ] Explainability tooling adapted per family, not assumed identical